Bound Value Capability

The Bound Value Validation design uses bound values to perform runtime validation that ensures application fail-safe processing and reliable log message generation. For example, by establishing a bound value of 64 characters length for log message string argument, the design checks and verifies that string arguments do not exceed 64 characters. This mitigates potential faults where string value is not properly NULL terminated.

Bound Value Validation ensures that the API functions are robust, reliable and prevent faults or abnormal behavior from impacting the application execution processing flow. For example, if any of the API fails validation it simply logs an error code and returns, allowing the application to continue normal processing.

Detailed programming examples on how some of the Bound Argument Validation works are provided in the `WandiSSAL-examples.c` program.

Growing list of Bound Value Validations and mitigation		
Definition	**Values**	**Benefits**
Select generation of output log message.	Generate both development and production log messages together or only production log message.	Reduce generating unnecessary development log messages used during code development in a production environment.
Number of format string specifiers types per log message	Maximum number of format string specifiers types allowed per message is 6.	Verify that log message arguments do not exceed an unreasonable number of arguments.
Different types of format string specifiers.	Different types of supported format string specifiers are %s, %d, %f, %c, %x, %p.	Prevent use of format spring specifiers not in the list and block attacker malicious specifier format. Prevent %n for memory overwrites. Align string specifier type with corresponding argument type.

<table>
<tr><td>Invalid string arguments for %s.</td><td>Prevent use of NULL, 0, or negative values as arguments for %s.</td><td>Avoid abnormal behavior or segmentation fault.</td></tr>
<tr><td>Using %s on a char causes segmentation fault.</td><td>Prevent the use of char for %s.</td><td>Avoid segmentation fault.</td></tr>
<tr><td>Using %c on a string could cause segmentation fault.</td><td>Prevent the use of string for %c</td><td>Avoid segmentation fault.</td></tr>
<tr><td>Length for string arguments.</td><td>Maximum length for %s string argument is 64 characters.</td><td>Avoid abnormal behavior due to unbounded, invalid string length or improperly NULL terminated string value.</td></tr>
<tr><td>Length for message format string specification.</td><td>Maximum length format string specification is 256 characters.</td><td>Avoid abnormal behavior due to unbounded, invalid string length or improperly NULL terminated string value.

Avoid string specification buffer overflow.</td></tr>
<tr><td>Subset of ASCII characters for format string specification and string argument.</td><td>Fixed subset of ASCII characters allowed: upper/lower case alphanumeric, blank and \\, %%, \’, \” and !#$&'()*+,-./:;<=>?@[]^_{~|} characters</td><td>Avoid characters not needed for message logging, such as non-printable characters.</td></tr>
<tr><td>Number of severity id=value pairs and length of value string.</td><td>Maximum number of severity levels is 4 and length of string value is 16 characters.</td><td>Allow use of valid pre-defined severity levels.</td></tr>
<tr><td>Number of logical message grouping id=value pairs and length of value string.</td><td>Maximum number of logical message groups is 32 and length of string value is 16 characters.</td><td>Allow use of valid pre-defined logical message groups.</td></tr>
<tr><td>New line at the end of log message.</td><td>Add \n at the end of each log message.</td><td>Never forget to add new line at the end of string specification.</td></tr>
</table>